



```
In [43]: import numpy as np
import pandas as pd
import os
import matplotlib.pyplot as plt
import time
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc, classification_report
```

```
In [44]: def load_data(folder_path="UCI HAR Dataset"):

    if not os.path.exists(folder_path):
        print(f"Папка '{folder_path}' не найдена!")
        print("Проверьте путь к папке с датасетом")
        return None, None, None, None, None

    X_train = pd.read_csv(os.path.join(folder_path, "train/X_train.txt"),
                          sep='\s+', header=None).values
    y_train = pd.read_csv(os.path.join(folder_path, "train/y_train.txt"),
                          header=None).values.flatten()

    X_test = pd.read_csv(os.path.join(folder_path, "test/X_test.txt"),
                         sep='\s+', header=None).values
    y_test = pd.read_csv(os.path.join(folder_path, "test/y_test.txt"),
                        header=None).values.flatten()

    activities = pd.read_csv(os.path.join(folder_path, "activity_labels.txt"),
                             sep='\s+', header=None)
    activity_names = dict(zip(activities[0], activities[1]))

    print(f"    Обучающая выборка: {X_train.shape}")
    print(f"    Тестовая выборка: {X_test.shape}")
    print(f"    Количество классов: {len(activity_names)}")
    print(f"    Активности: {list(activity_names.values())}")

    return X_train.astype(np.float32), y_train, X_test.astype(np.float32), y_t

FOLDER_PATH = "UCI HAR Dataset"

X_train, y_train, X_test, y_test, activity_names = load_data(FOLDER_PATH)

if X_train is None:
    print("\nнеправильный путь")
    exit()
```

The history saving thread hit an unexpected error (OperationalError('database or disk is full')).History will not be written to the database.

Обучающая выборка: (7352, 561)

Тестовая выборка: (2947, 561)

Количество классов: 6

Активности: ['WALKING', 'WALKING_UPSTAIRS', 'WALKING_DOWNSTAIRS', 'SITTING', 'STANDING', 'LAYING']

```
In [45]: def preprocess_data(X_train, X_test, n_features=100):
    mean = X_train.mean(axis=0)
```

```

std = X_train.std(axis=0)
std[std == 0] = 1
X_train_norm = (X_train - mean) / std
X_test_norm = (X_test - mean) / std

variances = np.var(X_train_norm, axis=0)
top_features = np.argsort(variances)[-n_features:]
X_train_reduced = X_train_norm[:, top_features]
X_test_reduced = X_test_norm[:, top_features]

print(f"  Train форма: {X_train_reduced.shape}")
print(f"  Test форма:  {X_test_reduced.shape}")

return X_train_reduced, X_test_reduced

X_train, X_test = preprocess_data(X_train, X_test, n_features=100)

print(f"\nРаспределение классов в TRAIN:")
unique, counts = np.unique(y_train, return_counts=True)
for u, c in zip(unique, counts):
    print(f"  {activity_names[u]}: {c}")

Train форма: (7352, 100)
Test форма:  (2947, 100)

```

Распределение классов в TRAIN:

```

WALKING: 1226
WALKING_UPSTAIRS: 1073
WALKING_DOWNSTAIRS: 986
SITTING: 1286
STANDING: 1374
LAYING: 1407

```

```

In [46]: class DecisionTree:
    def __init__(self, max_depth=3, min_samples_split=10, max_features=20):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.max_features = max_features
        self.tree = None
        self.features_subset = None

    def fit(self, X, y):
        n_features = X.shape[1]
        self.features_subset = np.random.choice(
            n_features,
            min(self.max_features, n_features),
            replace=False
        )
        self.tree = self._build_tree(X, y.astype(np.float32), depth=0)

    def _build_tree(self, X, y, depth):
        n_samples = len(y)

        if (depth >= self.max_depth or

```

```

        n_samples < self.min_samples_split or
        np.std(y) < 1e-5):
            return np.mean(y).astype(np.float32)

    best_mse = float('inf')
    best_feature = None
    best_threshold = None
    best_left_idx = None

    for feature in self.features_subset[:10]:
        values = X[:, feature]
        thresholds = np.percentile(values, np.linspace(10, 90, 9))

        for threshold in thresholds:
            left_idx = values <= threshold
            right_idx = ~left_idx

            if (np.sum(left_idx) < self.min_samples_split or
                np.sum(right_idx) < self.min_samples_split):
                continue

            y_left, y_right = y[left_idx], y[right_idx]
            mse = np.var(y_left) * len(y_left) + np.var(y_right) * len(y_r

            if mse < best_mse:
                best_mse = mse
                best_feature = feature
                best_threshold = threshold
                best_left_idx = left_idx

    if best_feature is None:
        return np.mean(y).astype(np.float32)

    left_tree = self._build_tree(X[best_left_idx], y[best_left_idx], depth
    right_tree = self._build_tree(X[~best_left_idx], y[~best_left_idx], de

    return (best_feature, best_threshold, left_tree, right_tree)

def predict(self, X):
    return np.array([self._predict_sample(x, self.tree) for x in X], dtype

def _predict_sample(self, x, node):
    if isinstance(node, (np.float32, float, np.float64)):
        return node
    feature, threshold, left, right = node
    if x[feature] <= threshold:
        return self._predict_sample(x, left)
    else:
        return self._predict_sample(x, right)

```

```

In [92]: class GradientBoosting:
    def __init__(self, n_estimators=50, learning_rate=0.1, max_depth=3,
                  min_samples_split=10, subsample=0.8):

```

```

self.n_estimators = n_estimators
self.learning_rate = learning_rate
self.max_depth = max_depth
self.min_samples_split = min_samples_split
self.subsample = subsample
self.models = []
self.n_classes = None

def _sigmoid(self, x):
    return 1 / (1 + np.exp(-np.clip(x, -10, 10)))

def fit(self, X, y):
    self.n_classes = len(np.unique(y))
    n_samples = len(X)

    print(f" Количество классов: {self.n_classes}")
    print(f" Деревьев на класс: {self.n_estimators}")
    print(f" Всего деревьев: {self.n_classes * self.n_estimators}")

    for class_id in range(self.n_classes):

        y_binary = (y == class_id).astype(np.float32)

        p = np.mean(y_binary)
        if p == 0:
            initial_pred = -5.0
        elif p == 1:
            initial_pred = 5.0
        else:
            initial_pred = np.log(p / (1 - p))

        F = np.full(n_samples, initial_pred, dtype=np.float32)
        models_for_class = []

        for i in range(self.n_estimators):
            if self.subsample < 1.0:
                idx = np.random.choice(n_samples, int(n_samples * self.subsample))
                X_sub = X[idx]
                grad_sub = (y_binary - self._sigmoid(F))[idx]
            else:
                X_sub = X
                grad_sub = y_binary - self._sigmoid(F)

            tree = DecisionTree(
                max_depth=self.max_depth,
                min_samples_split=self.min_samples_split
            )
            tree.fit(X_sub, grad_sub)

            F += self.learning_rate * tree.predict(X)
            models_for_class.append(tree)

```

```

        self.models.append(models_for_class)

    def predict_proba(self, X):
        n_samples = len(X)
        probas = np.zeros((n_samples, self.n_classes), dtype=np.float32)

        for class_id, models_for_class in enumerate(self.models):
            F = np.zeros(n_samples, dtype=np.float32)
            for tree in models_for_class:
                F += self.learning_rate * tree.predict(X)
            probas[:, class_id] = self._sigmoid(F)

        probas = probas / (probas.sum(axis=1, keepdims=True) + 1e-8)
        return probas

    def predict(self, X):
        probas = self.predict_proba(X)
        return np.argmax(probas, axis=1)

```

```

In [102... model = GradientBoosting(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=10,
    min_samples_split=20,
    subsample=0.8
)

start_time = time.time()
model.fit(X_train, y_train)
train_time = time.time() - start_time

print(f"  Время: {train_time:.1f} сек ({train_time/60:.1f} мин)")

```

Количество классов: 6
 Деревьев на класс: 100
 Всего деревьев: 600
 Время: 242.7 сек (4.0 мин)

```

In [103... y_test_pred = model.predict(X_test)
y_test_proba = model.predict_proba(X_test)

def calculate_metrics(y_true, y_pred):
    accuracy = np.mean(y_true == y_pred)

    classes = np.unique(y_true)
    precisions, recalls, f1s = [], [], []

    for c in classes:
        tp = np.sum((y_true == c) & (y_pred == c))
        fp = np.sum((y_true != c) & (y_pred == c))

```

```

        fn = np.sum((y_true == c) & (y_pred != c))

        prec = tp / (tp + fp) if (tp + fp) > 0 else 0
        rec = tp / (tp + fn) if (tp + fn) > 0 else 0
        f1 = 2 * prec * rec / (prec + rec) if (prec + rec) > 0 else 0

        precisions.append(prec)
        recalls.append(rec)
        f1s.append(f1)

    return accuracy, np.mean(precisions), np.mean(recalls), np.mean(f1s)

test_acc, test_prec, test_rec, test_f1 = calculate_metrics(y_test, y_test_pred)

print(f"\nРезультаты на тестовой выборке:")
print(f" Accuracy: {test_acc:.4f}")
print(f" Precision: {test_prec:.4f}")
print(f" Recall: {test_rec:.4f}")
print(f" F1-score: {test_f1:.4f}")

```

Результаты на тестовой выборке:

```

Accuracy: 0.7441
Precision: 0.7166
Recall: 0.7560
F1-score: 0.7338

```

```

In [104... class_names = [activity_names[i+1] for i in range(6)]
y_test_bin = label_binarize(y_test, classes=range(6))

plt.figure(figsize=(10, 8))

for i in range(6):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_test_proba[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f'{class_names[i]} (AUC = {roc_auc:.3f})')

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-кривые для всех классов')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.show()

```

ROC-кривые для всех классов

